

besir 개발 로드맵

5개 서비스(be healthy / on-time / full / fun / rich sir) 단계별 실행 계획

작성 기준일 2026-09-10 · Claude Code (Pro 플랜) 세션 단위 일자별 작업 리스트

소스: [besir 사업계획서.pdf](#) + [STATUS.md](#) + 개발 메모리(2026-06-29~09-09).

목적: 사업계획서의 5개 서비스(be healthy/on-time/full/fun/rich sir)를 어떤 순서로, 어떤 파일/API 단위로 만들지 정하고, Claude Code(Pro 플랜)가 매일 새 세션에서 그대로 실행할 수 있을 만큼 구체적인 작업 리스트로 쪼갬다.

이 문서는 앞으로 진행되는 besir 세션들의 1차 참조 스펙이다 — 매일 작업 전 이 파일에서 해당 Day 항목을 읽고 시작하고, 계획이 실제와 달라지면(설계 변경, API 제약 발견 등) 이 파일을 그 자리에서 갱신한다.

1. 현재 상태 요약 (2026-09-10 “Besir 앱 세션 동기화” 세션 반영 갱신)

아래는 세션 [Besir 앱 세션 동기화](#)(로컬 세션, cwd ~/Projects/besir)의 실제 진행 내역을 읽어 갱신한 최신 상태다. Phase 0(AI 파이프라인)은 최초 계획과 다른 아키텍처로 이미 완료됐고, 그 사이 계획에 없던 be on-time sir 고도화 작업이 상당량 추가로 진행됐다.

- be on-time sir: 완성 + 고도화 진행 중. 일정 등록→이동시간 역산→출발알람→카카오맵 경로선→구글 캘린더 양방향 동기화까지 안정 동작. 여기에 더해 즐겨찾기 장소, 출퇴근 왕복+점심 이동을 포함한 복합 반복 일정, AI 대화 기록 영구 저장, 월/주/일 통합 스와이프 캘린더 UI(드래그로 5분단위 일정 이동 포함)까지 이번 세션에서 추가됨(\$6 Phase 0.5 참고).
- AI 파이프라인(카카오톡 등에서 공유 텍스트/이미지 → 일정 자동 등록) — 완료, 아키텍처가 최초 계획과 다름:
- ~~Anthropic Claude API 배포~~ → Google Gemini 시도 후 폐기 → 최종 Cloudflare Workers AI로 확정([@cf/mistralai/mistral-small-3.1-24b-instruct](#), [proxy/src/index.js](#)의 [/ai/chat](#) 라우트, [env.AI](#) 바인딩). 별도 API 키 불필요, Cloudflare 자체 인프라에서 실행돼 외부 호출이 없다 — Gemini에서 겪은 “Cloudflare 데이터센터 IP가 무료 티어에 지역 무관 차단당함(성공률 10~50%로 들쭉날쭉)” 문제가 구조적으로 사라짐(실측 8/8 성공).
- 와이어 포맷은 의도적으로 Gemini [generateContent](#) 형식을 그대로 유지 — 앱([AIAssistant.swift](#))은 백엔드가 뭐든 몰라도 되고, 나중에 GPT/Claude로 다시 바꿀 때도 프록시의 번역 함수([proxyWorkersAI/toGeminiShape](#))만 고치면 됨. 이 계약을 앞으로도 지킬 것.
- 이미지 입력도 [inlineData](#)(base64) → Workers AI [image_url](#) content part로 프록시가 번역해 지원(텍스트+이미지 둘 다 동작 확인).
- [ANTHROPIC_KEY](#) 배포는 더 이상 블로커가 아님(하지 않아도 됨) — 프록시 주석에 “(구) LLM 대화형 일정 등록용, 현재 미사용”으로 남아 있고, 나중에 GPT/Claude로 전환하고 싶을 때를 위한 옵션으로만 존재.
- Share Extension(카카오톡/사진 등에서 공유 → besir로 전달) — 구현 완료, 실기기 라이브 테스트로 최소 1회 이상 성공 확인:

- 최초 계획한 "URL scheme 핸드오프"가 아니라 App Group 파일 큐 방식으로 구현됨: `ShareExtension/ShareViewController.swift`(공유 시트에서 텍스트/이미지 추출) → `Shared/SharedInbox.swift`(group.com.iseongmin.besir 앱그룹 컨테이너의 `pendingShares.json`에 큐잉) → `Shared/App.swift`가 앱 포그라운드 진입(`scenePhase == .active`) 시 `SharedInbox.drain()`으로 꺼내 `AIAssistant`에 전달 → `AIChatView` 자동 오픈.
- 실기기 라이브 테스트에서 공유→일정 실제 등록까지 성공 확인됨. 다만 확인 메시지를 받아오는 2차 API 호출만 실패해 성공했는데도 "처리 중 문제가 생겼어요"로 잘못 표시되는 버그가 있었고 → 이미 수정됨(성공한 결과는 그대로 보여주고 에러로 표시하지 않도록).
- 카카오톡 자체의 "말풍선 꼭 눌러 ChatGPT로 보내기" 메뉴는 카카오톡-OpenAI 특별 제휴라 일반 개발자는 접근 불가로 확인됨 → besir는 iOS 표준 공유 시트(공유 → besir 아이콘 선택) 경로로 동작.
- 진행 중(세션 마지막 시점, 아직 실기기 재확인 전): 캘린더 UI의 두 가지 버그를 직전에 고쳐 아이폰에 설치까지 마쳤으나, 사용자의 라이브 재테스트 확인은 아직 못 받은 상태로 세션이 끝남 — ① 월간 캘린더 좌우 스와이프 시 프레임 드랍(원인: 42칸×3페이지마다 전체 배열 스캔 → `Store.daysWithSchedule` O(1) 캐시로 수정), ② 블록을 탭해도 상세정보 시트가 안 뜨던 문제(원인: `NavigationSplitView` detail 컬럼 전환 방식이 아이폰 좁은 화면에서 커스텀 제스처의 `selection` 변경만으론 안 먹힘 → `.sheet`로 변경). 다음 세션 최우선: 이 두 수정이 실기기에서 실제로 되는지 확인부터 하고 Phase 1으로 넘어갈 것.
- 뒤로 미룬 것(사용자가 명시적으로 나중에 미룸): AI 백엔드를 Workers AI → GPT/Claude로 다시 바꾸는 작업. "나중에 할게"라고 명시적으로 보류함 — 사용자가 다시 꺼내기 전엔 먼저 제안하지 말 것.
- be healthy / be full / be fun / be rich sir: 여전히 코드 없음. 사업계획서상 "초안 기획" 단계 그대로 — 데이터 모델부터 새로 설계해야 함(아래 Phase 1부터 시작).

2. 사업계획서 기준 우선순위 근거

8장(향후 계획)이 명시: "MVP 범위 확정: 우선 be on-time sir(일정) + be full sir(음식) 연동을 핵심으로 한 최소 기능 제품 개발". 4장(연계 구조)을 보면 be on-time sir가 다른 4개 서비스 전부와 연결되는 허브(운동 장소·음식점·여가활동을 전부 "일정+위치" 기반으로 추천)이므로, 이미 완성된 be on-time sir를 축으로 다른 서비스를 하나씩 "일정에 연동되는 서브 서비스"로 붙여나가는 순서가 사업계획서와도, 기존 코드 재사용 관점에서도 맞다.

우선순위: (0) AI 파이프라인 완성 → (1) be full sir → (2) be healthy sir → (3) be rich sir → (4) be fun sir → (5) 통합/베타. 근거는 각 Phase 서두에 기술.

3. 전체 아키텍처에 새로 추가되는 것들 (사전 결정)

새 서비스마다 매번 재논의하지 않도록, 이 로드맵 전체에 적용되는 공통 설계를 먼저 고정한다.

- 데이터 저장: 기존 `events.json`과 같은 패턴으로 서비스별 JSON 파일을 `~/Library/Application Support/besir/(macOS)/` 앱 컨테이너(iOS)에 추가. 예: `meals.json`, `expenses.json`. `Codable` 구조체 + `Store`에 배열 하나씩 추가하는 기존 관례를 그대로 따름(새 영속화 레이어를 만들지 않음).
- `Store`를 계속 단일 허브로 사용: 지금까지 `Store`(`ObservableObject`)에 `events`뿐 아니라 `meals`, `healthGoals`, `expenses`, `funActivities`를 나란히 추가. 서비스 간 연계(4장 표)는 전부 "A 서비스가 `Store`의 B 서비스 배열을 읽어서 추천에 활용"하는 형태로 구현 — 별도 이벤트버스/메시징 불필요.

- AI 확장은 톨 추가 방식 유지: `AIAssistant`의 기존 톨 루프(`create_schedule/create_recurring_schedule`)에 새 톨(`recommend_meal, log_expense` 등)을 계속 추가하는 구조 유지. 새 서비스마다 별도 AI 클래스를 만들지 않음. 백엔드는 현재 Cloudflare Workers AI(`mistral-small-3.1-24b-instruct`, 프록시 `/ai/chat`)이고 와이어 포맷은 Gemini `generateContent` 형식으로 고정돼 있다 — 새 톨을 추가할 때도 이 포맷(함수 선언은 대문자 타입 `"STRING"/"OBJECT"` 등, 프록시가 `lowercaseSchemaTypes`로 자동 소문자 변환)을 그대로 따르면 되고, 백엔드를 나중에 GPT/Claude로 바꿔도 앱 코드는 안 건드려도 된다.
- 카카오 로컬 API 카테고리 재사용: `PlaceSearch`가 이미 프록시 `/kakao/local/keyword`를 호출 중. 카카오 로컬 API는 `category_group_code` 파라미터로 `FD6`(음식점), `CE7`(카페) 필터를 지원 — be full sir의 "맛집 추천"은 이 파라미터만 추가하면 새 API 없이 구현 가능(Day 6에서 확인).
- HealthKit/외부 연동은 각 Phase 시작 전 "조사 전용 Day"를 하나 둔다 — 실제 API 문서를 그 시점에 확인하고 설계를 확정된 뒤 구현일에 들어간다(사업계획서에 없는 세부는 지금 확정하지 않음).

4. Claude Pro 플랜으로 일할 때의 작업 단위 원칙

- 하루 = 새 세션 1~2개, 한 세션 = 아래 표의 한 Day 항목 전체(그 이상 묶지 않음). 컨텍스트 0에서 시작해도 "무엇을, 어느 파일에, 어떻게" 프롬프트만 보고 끝낼 수 있게 아래 표에 파일 경로/함수 시그니처까지 명시했다.
- 조사(Research) Day와 구현(Implement) Day를 분리했다 — 외부 API 제약을 그날 알아내고, 구현은 다음 세션에서 결과를 그대로 프롬프트에 넣어 시작한다.
- 한 Day당 새로 만들거나 크게 고치는 파일은 최대 3~4개로 제한(그 이상이면 다음 Day로 분리). 리팩터링은 계획에 없는 한 하지 않는다(기존 CLAUDE 지침과 동일하게 범위 최소화).
- 매일 끝에 완료 기준(빌드 성공 + 시뮬레이터에서 1회 동작 확인)을 두어 하루치가 실패해도 다음 Day로 깨끗하게 넘어간다.
- 날짜(Day 1, 2, 3...)는 순서 보장용이지 캘린더 마감이 아니다. 막히면 다음 세션으로 그대로 밀면 된다.

5. 품질 게이트 — 코드 검사 / 실기기 테스트 / 안정화 (매 Phase 공통 기준)

각 Phase는 기능 구현 Day들 뒤에 코드 검사 → 실기기 테스트 → 안정화 3일을 고정으로 붙인다. 이 3일의 세부 체크리스트는 아래 공통 기준을 기본으로 삼고, 각 Phase 표에는 그 Phase에만 해당하는 항목만 추가로 적는다(공통 항목을 매번 반복 기재하지 않음).

5-1. 코드 검사(Code Review) 체크리스트 — 공통

- 강제 언래핑(!)·강제 캐스팅(as!) 남용 없는지 — 실패 가능한 지점은 `guard let/try?/if let`로 처리했는지
- 네트워크/API 실패 시 UI가 무한 로딩이 아니라 명시적 실패 상태를 보여주는지(기존 `failedEstimateBlockView` 패턴 참고)
- 새 모델의 영속화가 `events.json`과 같은 `Codable` + 파일저장 패턴을 그대로 따르는지, Store에 `@Published`로 선언됐는지
- `async` 클로저에서 Store/AIAssistant 참조 순환 방지를 위해 `[weak self]`가 필요한 곳에 붙어있는지
- UI를 갱신하는 코드가 메인 스레드(`@MainActor` 등)에서 실행되는지

- 새 기능이 이전 Phase가 의존하는 기존 함수 시그니처(`Store.addEvent`, `AIAssistant`의 기존 톨 스키마 등)를 깨지 않았는지
- 카카오/ODsay/Workers AI 호출이 불필요하게 반복되지 않는지(캐시 가능한 값은 캐시 — `Store.daysWithSchedule`처럼)
- 새로 추가한 AI 톨이 Gemini 형식 스키마 계약(§3)을 그대로 따르는지(대문자 타입명, 프록시의 `lowercaseSchemaTypes` 의존)
- 한 Day당 3~4개 파일 제한(§4)을 넘겨 손덴 파일이 있으면 이유가 타당한지

5-2. 실기기 테스트 체크리스트 — 공통 절차 (Phase 전용 시나리오는 각 Phase 표에 명시)

- iOS 시뮬레이터 + macOS 둘 다 빌드 성공
- 아이폰(UDID `8D9B807B-F874-5AD8-A62C-BC1731A31A1F`)에 설치 및 실행
- 이번 Phase 신규 기능의 happy path 1회 성공
- 이번 Phase 신규 기능의 엣지 케이스(네트워크 없음/빈 검색 결과/잘못된 입력 등) 최소 1개 확인
- 회귀 확인(항상 실행): be on-time sir 핵심 흐름 — 일정 수동 등록→출발 알람→구글 캘린더 동기화, 그리고 카카오톡 공유→AI 파싱→일정 자동 등록 — 각 최소 1회씩 재확인해 이전 Phase가 망가지지 않았는지 확인
- 이 Day에서 발견한 문제는 그 자리에서 고치지 않고 다음 “안정화” Day로 넘긴다(테스트와 수정을 같은 세션에 섞지 않아 각 세션이 명확히 끝나게 함)

5-3. 안정화(Stabilization) 기준 — 공통

- 코드 검사·실기기 테스트에서 발견된 문제를 전부 수정하거나, 수정하지 않기로 했으면 §7(리스크)에 “알려진 이슈”로 명시적으로 남긴다
- 회귀로 발견된 기존 기능 문제는 최우선으로 수정(새 기능보다 우선순위 높음)
- `STATUS.md`에 이번 Phase 완료 사실 반영
- 커밋

6. 일자별 실행 리스트

Phase 0 — AI 카카오톡 공유 파이프라인 완성 — 완료 (당초 계획과 다른 아키텍처로 구현됨)

당초 계획: Anthropic Claude API 키를 프록시에 배포 + URL scheme 기반 Share Extension 핸드오프.

실제 구현(더 나은 방향으로 확정됨): Gemini를 시도했다가 Cloudflare 엣지 IP가 Gemini 무료 티어에 지역 무관 차단당하는 문제(성공률 10~50%)를 겪고 Cloudflare Workers AI(`@cf/mistralai/mistral-small-3.1-24b-instruct`)로 완전 전환 — 외부 호출 자체가 없어져 그 문제가 구조적으로 사라짐. Share Extension은 URL scheme이 아니라 App Group 파일 큐(`SharedInbox.swift`)로 구현됨. 상세는 §1 참고.

항목	상태	비고
AI 백엔드 확보(무료, 안정적)	완료	Cloudflare Workers AI, /ai/chat 라우트, 키 관리 불필요
Share Extension(텍스트)	완료, 실기기 성공 확인	App Group 큐 + 앱 포그라운드 drain 방식
Share Extension(이미지)	완료	inlineData → Workers AI image_url로 프록시가 번역, 모델이 비전 지원(Mistral Small 3.1)
파싱 실패/2차 API 실패 시 오픈시 버그	수정 완료	등록 자체는 성공했는데 확인 메시지 호출만 실패해 에러로 잘못 뜨던 문제
다음 세션에서 재확인 필요	미확인	월간 캘린더 스와이프 렉 수정, 탭→상세정보 시트 전환 수정 — 둘 다 아이폰에 설치까지는 됐고 실제 라이브 재테스트만 남음(\$6 Day 5)

Phase 0.5 — be on-time sir 고도화 (Phase 0과 병행 진행, 완료)

원래 계획엔 없었지만 AI 파이프라인 작업 중 사용자 요청으로 함께 진행되어 완료된 항목들 — be on-time sir 자체의 완성도를 높이는 작업이라 순서상 문제 없음.

항목	내용	대상 파일
즐거찾기 장소	“집” 등 자주 쓰는 장소 저장 → 일정 추가 화면에서 칩으로 빠른 선택, AI가 목적지/출발지명을 즐겨찾기와 우선 매칭	Shared/FavoritesView.swift
복합 반복 출퇴근 일정	“매주 평일 9시~18시 교육, 점심 1~2시” 같은 표현 → 출근+퇴근 왕복 이동을 자동 생성, 점심도 “근처 식당” 등 다른 장소로 명시하면 왕복 이동 생성(같은 건물이면 생성 안 함), 출발지·복귀 여부·이동수단이 애매하면 AI가 등록 전에 먼저 되물음	Shared/AIAssistant.swift, Shared/Store.swift(addRecurringActivities), Shared/Models.swift(ActivityBlock, ScheduleAnchor)
AI 대화 기록 영구 저장	재설치해도 대화 이력이 사라지지 않도록 ai_history.json으로 저장/로드	Shared/AIAssistant.swift
캘린더 UI 전면 개편	일간/주간/월간 탭 제거 → 월간 그리드 기본, 날짜 탭하면 그 날 시간표+상단 주간 스트립, 좌우 스와이프로 날짜/월 이동(실시간 손가락 추적 애니메이션), 블록 꼭 눌러 드래그로 5분 단위 재조정(반복 일정이면 전체/이 일정만 이동 확인 다이얼로그), 활동 블록 이동 시 연계된 이동 블록도 같이 이동, 이동 블록만 옮기면 활동은 고정하고 버퍼만 조정	Shared/ContentView.swift(대규모 재작성), Shared/Store.swift
월간 스와이프 렉 수정	Store.daysWithSchedule(Set<DateComponents> 캐시)로 O(n)→O(1)	Shared/Store.swift, Shared/ContentView.swift
탭→상세정보 안 뜨던 버그 수정	NavigationSplitView detail 컬럼 전환 대신 .sheet 사용	Shared/ContentView.swift

남은 리스크: 위 표의 마지막 두 수정(스와이프 렉, 탭 상세정보)은 세션이 끝나는 시점에 아이폰 설치까지만 마쳤고 사용자의 라이브 재확인 피드백은 아직 못 받았다.

Day	작업	대상 파일	완료 기준
5	(최우선, 다음 세션 시작점) 최근 설치된 빌드로 실기기 라이브 재테스트: ① 블록 위에서 빠른 스크롤이 되는지, ② 이동/활동 블록 탭 시 상세정보 시트가 뜨는지, ③ 롬프레스-드래그 재조정과 좌우 스와이프 내비게이션에 회귀가 없는지. 문제 있으면 그 자리에서 수정, 없으면 바로 Phase 1 (Day 6)으로 진행	Shared/ContentView.swift	3가지 상호작용 모두 실기기에서 정상 동작 확인

Phase 1 — be full sir MVP (식사 추천) — Day 6~11

왜 다음인가: 사업계획서가 명시한 2번째 MVP 축이며, **PlaceSearch**(카카오 로컬 검색)가 이미 있어 신규 외부 연동 없이 "맛집 추천"을 구현 가능 — 가장 리스크 낮은 신규 서비스.

Day	작업	대상 파일	완료 기준
6	데이터 모델 설계(구현 최소, 설계 위주): <code>Models.swift</code> 에 struct <code>MealLog: Codable, Identifiable { id: UUID; category: MealCategory /* .diningOut, .delivery, .cooking */; title: String; place: Place?; estimatedCost: Int?; scheduledEventId: UUID?; loggedAt: Date }</code> 추가. <code>Store</code> 에 <code>@Published var meals: [MealLog] + meals.json</code> 로드/저장 함수(기존 <code>events</code> 저장 패턴 그대로 복사)	Shared/Models.swift, Shared/Store.swift	빌드 성공, 빈 배열로 저장/로드 왕복 확인
7	외식 추천 구현: <code>PlaceSearch</code> (또는 <code>DirectionsService</code> 와 같은 레벨의 신규 <code>MealSuggestionService</code>)에서 카카오 로컬 키워드검색 호출 시 <code>category_group_code=FD6</code> 파라미터 추가 → 일정 상세(<code>EventDetailView</code>)에 "주변 맛집 추천" 섹션 추가, 일정의 <code>destination</code> 좌표 기준 반경 검색	Shared/PlaceSearch.swift 또는 신규 Shared/MealSuggestionService.swift, Shared/EventDetailView.swift	일정 상세에서 목적지 주변 맛집 리스트 3~5개 표시 확인
8	(조사 Day, 구현 없음) 배달/요리 카테고리 조사: 요기요·배민 공식 오픈 API가 개인 개발자에게 열려있는지 웹 검색으로 확인 → 없을 경우 URL scheme 딥링크(<code>yogyo://</code> , <code>baemin://</code> 등 존재 여부) 방식으로 결론. 조사 결과를 이 <code>plan.md</code> 의 Day 9 항목 아래에 메모로 추가	<code>plan.md</code> (조사 결과 기록)	배달 카테고리 구현 방식이 확정되어 다음 세션 프롬프트에 바로 쓸 수 있음
9	AI 채팅 확장: <code>AIAssistant</code> 에 신규 툴 <code>recommend_meal(budget?: Int, category?: string)</code> 추가 — 현재 위치 + 최근 <code>meals</code> 이력 + (있다면) <code>expenses</code> 예산 정보를 시스템 프롬프트에 주입해 자연어 "저녁 뭐 먹지" 요청 처리 → Day 7의 추천 로직 재사용. Gemini 형식 툴 선언 계약 유지(\$3)	Shared/AIAssistant.swift	AI 채팅에 "저녁 뭐 먹을까"로 물으면 추천 응답 1회 성공

Day	작업	대상 파일	완료 기준
10	be on-time sir 연동: 식사 추천을 사용자가 선택하면 기존 <code>Store.addEvent</code> 파이프라인으로 "식사 일정"(제목=식당명, mode 임의/도보 기본) 등록 + <code>MealLog.scheduledEventId</code> 연결	<code>Shared/Store.swift</code> , <code>Shared/MealSuggestionService.swift</code>	추천 선택 → 캘린더에도 자동 등록되는 것까지 확인
11	코드 검사(\$5-1 공통 + 아래 전용 항목)	<code>Shared/Models.swift</code> , <code>Shared/Store.swift</code> , <code>Shared/MealSuggestionService.swift</code> , <code>Shared/AIAssistant.swift</code>	체크리스트 전 항목 확인, 발견 사항은 Day 13로 넘김
12	실기기 테스트(\$5-2 공통 + 아래 전용 시나리오)	-	체크리스트 전 항목 실행, 결과 기록
13	안정화(\$5-3 공통)	<code>STATUS.md</code>	be full sir MVP 완료, 회귀(기존 be on-time sir 기능) 이상 없음

Phase 1 전용 코드 검사 항목(Day 11):

- `MealSuggestionService`가 `PlaceSearch/DirectionsService`와 같은 프록시 호출 관례(`config.proxyRequest`)를 따르는지
- `category_group_code=FD6` 파라미터가 기존 키워드 검색 쿼리 파라미터와 충돌 없이 추가됐는지
- `recommend_meal` 톨의 예산/이력 주입이 시스템 프롬프트 길이를 과도하게 늘리지 않는지(최근 N건으로 제한했는지)

Phase 1 전용 실기기 테스트 시나리오(Day 12):

- 일정 상세에서 "주변 맛집 추천" 3~5개가 실제로 표시되는지(목적지 좌표 기준)
- AI 채팅에 "저녁 뭐 먹지"로 물었을 때 추천이 오는지, 즐겨찾기/최근 식사 이력이 있을 때와 없을 때 둘 다
- 추천 선택 → `meals.json`에 기록 + 캘린더에 "식사 일정"으로 자동 등록되는지
- 카카오 로컬 검색 결과가 0건일 때(외곽 지역 등) 빈 상태 UI가 크래시 없이 뜨는지

Phase 2 — be healthy sir MVP — Day 14~20

왜 다음인가: HealthKit은 애플 표준 프레임워크라 외부 파트너십 없이 바로 시작 가능하고, be full sir가 만든 식사 데이터를 곧바로 영양소 계산에 재사용할 수 있어 의존성 순서상 자연스럽다.

Day	작업	대상 파일	완료 기준
14	HealthKit 연동 기초: <code>Info.plist</code> 에 <code>NSHealthShareUsageDescription</code> 추가, 프로젝트에 HealthKit capability(<code>project.yml</code> entitlements), <code>HealthKitService.requestAuthorization() + fetchTodaySteps()/fetchTodayActiveEnergy()</code> (HKStatisticsQuery) 구현	<code>Shared/HealthKitService.swift</code> (신규), <code>project.yml</code>	시뮬레이터/실기기에서 오늘 걸음수 콘솔 출력 확인(시뮬레이터는 Health 앱에 더미 데이터 필요)

Day	작업	대상 파일	완료 기준
15	건강 목표 UI: Models.swift에 struct HealthGoal: Codable { targetCalories: Int; targetActiveMinutes: Int }, HealthDashboardView(목표 입력 + Day 14 데이터로 달성률 프로그레스바)	Shared/Models.swift, Shared/HealthDashboardView.swift(신규)	목표 입력 후 달성률(%) 표시 동작
16	be full sir 연동: MealLog에 estimatedCalories: Int? 필드 추가(1차는 AI 톨 recommend_meal 응답에 칼로리 추정치를 포함시켜 채움 — 별도 영양 DB 연동은 이번 스코프 제외), HealthDashboardView에 오늘 식사 칼로리 합산 표시	Shared/Models.swift, Shared/AIAssistant.swift, Shared/HealthDashboardView.swift	식사 기록 후 대시보드에 칼로리 합산 반영
17	be on-time sir 연동: 활동량 목표 미달 시(Day 14 데이터 기준) Store에 “운동 일정 제안” 배너 → 수락 시 기존 addEvent로 캘린더 등록	Shared/Store.swift, Shared/ContentView.swift	목표 미달 조건에서 제안 배너 → 수락 시 일정 등록 확인
18	코드 검사(\$5-1 공통 + 아래 전용 항목)	Shared/HealthKitService.swift, Shared/Models.swift, Shared/HealthDashboardView.swift	체크리스트 전 항목 확인, 발견 사항은 Day 20으로 넘김
19	실기기 테스트(\$5-2 공통 + 아래 전용 시나리오, HealthKit은 반드시 실기기 또는 Health 앱에 더미 데이터 넣은 시뮬레이터에서)	-	체크리스트 전 항목 실행, 결과 기록
20	안정화(\$5-3 공통)	STATUS.md	be healthy sir MVP 완료

Phase 2 전용 코드 검사 항목(Day 18):

- HealthKit 권한 거부 시 앱이 크래시 없이 “권한 없음” 상태를 보여주는지
- HKStatisticsQuery 콜백이 메인 스레드로 안전하게 튀는지(DispatchQueue.main/@MainActor)
- 운동 일정 제안 배너가 매 앱 실행마다 중복으로 뜨지 않는지(하루 1회 등 제한 로직)

Phase 2 전용 실기기 테스트 시나리오(Day 19):

- Health 앱에 걸음수/활동 칼로리 더미 데이터를 넣은 뒤 대시보드에 반영되는지
- 목표 미달성 상태에서 “운동 일정 제안” 배너 → 수락 시 캘린더에 실제 등록되는지
- 식사 기록 후 칼로리 합산이 대시보드에 즉시 반영되는지
- HealthKit 권한을 거부한 상태로 앱을 켜를 때 나머지 기능(be on-time/full sir)이 영향받지 않는지

Phase 3 — be rich sir MVP (수동 입력 기반, 축소 스코프) — Day 21~26

왜 스코프를 줄였나: 토스 등 금융 데이터는 마이데이터 사업자 등록 없이 공식 API 연동이 사실상 불가능하다(조사 없이도 일반적으로 알려진 제약) → 처음부터 "수동 입력 + 분석/제안"으로 범위를 좁혀 실현 가능하게 한다.

Day	작업	대상 파일	완료 기준
21	데이터 모델: <code>struct Expense: Codable, Identifiable { id: UUID; amount: Int; category: ExpenseCategory; date: Date; linkedEventId: UUID?; linkedMealId: UUID? },</code> Store에 <code>expenses</code> 배열 + <code>expenses.json</code> 영속화(기존 패턴 그대로)	Shared/Models.swift, Shared/Store.swift	빌드 성공, 저장/로드 왕복 확인
22	소비 입력 UI + 리포트: <code>ExpenseListView</code> (입력 폼) + <code>ExpenseReportView</code> (카테고리별 합계 파이/바 차트 또는 단순 리스트, 이번 주 목표 대비 초과 여부)	Shared/ExpenseListView.swift, Shared/ExpenseReportView.swift(신규)	지출 입력 → 카테고리별 합계 확인
23	be full/be on-time 연동: 식사(<code>MealLog.estimatedCost</code>)·이동 관련 지출을 <code>Expense</code> 로 자동 제안(사용자 확인 후 등록, 자동 확정은 하지 않음 — 금액 데이터는 틀리면 신뢰를 잃는 영역이므로 확인 단계 필수)	Shared/Store.swift	식사 등록 후 지출 자동 태깅 제안 → 확인 시 등록
24	코드 검사(\$5-1 공통 + 아래 전용 항목)	Shared/Models.swift, Shared/ExpenseListView.swift, Shared/ExpenseReportView.swift	체크리스트 전 항목 확인, 발견 사항은 Day 26으로 넘김
25	실기기 테스트(\$5-2 공통 + 아래 전용 시나리오)	-	체크리스트 전 항목 실행, 결과 기록
26	안정화(\$5-3 공통)	STATUS.md	be rich sir MVP 완료

Phase 3 전용 코드 검사 항목(Day 24):

- 금액(`amount: Int`) 입력 검증 — 음수/빈 값이 그대로 저장되지 않는지
- 자동 태깅 제안이 "제안"에 그치고 사용자 확인 없이 `Expense`가 바로 생성되지 않는지(\$3 안전 원칙)
- `linkedEventId/linkedMealId`가 삭제된 일정/식사를 참조하는 경우(dangling) 리포트 화면이 크래시하지 않는지

Phase 3 전용 실기기 테스트 시나리오(Day 25):

- 지출을 수동으로 입력 → 카테고리별 합계 리포트에 즉시 반영되는지
- 식사/이동 일정 등록 후 지출 자동 태깅 제안이 뜨고, 확인 시에만 등록되는지(취소 시 등록 안 되는지도 확인)
- 이번 주 목표 금액을 초과했을 때 리포트에 초과 표시가 뜨는지
- 연결된 일정/식사를 삭제한 뒤 지출 리포트를 열어도 정상 동작하는지(dangling 참조 케이스)

Phase 4 — be fun sir MVP — Day 27~31

왜 마지막 신규 서비스인가: 사업계획서상 우선순위가 가장 낮고, 실제 유의미한 추천(인터파크/야놀자 연동)은 외부 제휴가 필요해 리스크가 높다 — 우선 규칙 기반 최소 버전만 구현.

Day	작업	대상 파일	완료 기준
27	여가활동 추천 로직(규칙 기반, 외부 API 없이): 오늘 일정의 빈 시간대 + 현재 위치 기준 <code>category_group_code=CT1</code> (문화시설)/ <code>AT4</code> (관광명소) 카카오 로컬 검색으로 후보 제시	<code>Shared/FunSuggestionService.swift</code> (신규)	빈 시간대에 근처 여가 후보 리스트 표시
28	be on-time sir 연동: 추천 항목 선택 시 기존 <code>addEvent</code> 로 일정 등록	<code>Shared/Store.swift</code>	여가 일정 등록 확인
29	코드 검사(\$5-1 공통 + 아래 전용 항목)	<code>Shared/FunSuggestionService.swift</code> , <code>Shared/Store.swift</code>	체크리스트 전 항목 확인, 발견 사항은 Day 31로 넘김
30	실기기 테스트(\$5-2 공통 + 아래 전용 시나리오)	-	체크리스트 전 항목 실행, 결과 기록
31	안정화(\$5-3 공통)	<code>STATUS.md</code>	be fun sir MVP 완료

Phase 4 전용 코드 검사 항목(Day 29):

- “빈 시간대” 계산이 자정을 넘기는 일정이나 겹치는 일정에서도 올바른지
- 카카오 로컬 검색 결과가 0건일 때 빈 상태 UI 처리

Phase 4 전용 실기기 테스트 시나리오(Day 30):

- 오늘 일정 사이 빈 시간대가 있을 때 여가 후보가 실제로 뜨는지
- 후보 선택 → 캘린더에 등록되는지
- 일정이 하루 종일 꽉 찬 날에는 추천이 뜨지 않거나 “추천 없음”으로 명확히 표시되는지

Phase 5 — 통합 대시보드 & 베타 준비 — Day 32~38

Day	작업	대상 파일	완료 기준
32	메인 화면 재설계: <code>ContentView</code> 를 “오늘의 시간표 + 5개 서비스 핵심 지표 카드”(사업계획서 3장 표 그대로 — 목표 칼로리/운동량, 오늘 사용 금액/목표 금액 등) 레이아웃으로 개편(이미 완료된 월/주/일 스와이프 캘린더 UI 위에 지표 카드만 추가하는 형태)	<code>Shared/ContentView.swift</code>	홈 화면에서 5개 지표 카드 확인

Day	작업	대상 파일	완료 기준
33	서비스간 연계 누락 점검: 사업계획서 4장 표(5개 표 전체)를 체크리스트로 놓고 이번 로드맵에서 구현 안 된 연결(예: be fun sir 소비 자동 기록, be rich sir → be healthy sir 제안) 확인 및 우선순위 높은 것만 보완	해당 파일들	체크리스트 대비 구현물 문서화
34	프록시 보안 강화: Cloudflare WAF 레이트리밋 규칙 추가, APP_TOKEN 로테이션 절차 문서화. Workers AI 자체의 무료 티어 일일 한도도 이 시점에 확인(사용량이 늘어난 뒤이므로 실측 필요)	proxy/wrangler.toml , Cloudflare 대시보드	레이트리밋 1건 적용 확인
35	베타 준비: 테스트 시나리오 문서 정리, 파트너십 후보 1곳(예: 카카오/구글 외 구글Fit 또는 삼성헬스) 컨택 초안 작성	STATUS.md 또는 별도 문서	베타 착수 가능 상태
36	전체 코드 감사: §5-1 체크리스트를 프로젝트 전체(Phase 0~4에서 만든 모든 신규 파일)에 다시 적용 — 개별 Phase에서는 놓쳤을 수 있는 서비스 간 상호작용(예: Store 에 배열이 5개나 쌓이면서 생긴 로딩 순서 문제) 위주로 확인	프로젝트 전체	감사 결과 문서화(발견 시 Day 38로)
37	전체 실기기 회귀 테스트: 5개 서비스 각각의 핵심 시나리오(§5-2의 Phase별 시나리오 전부)를 한 기기에서 순서대로 실행 — 개별 Phase 테스트 때는 없던 “여러 서비스 동시 사용” 상태(예: 일정 많고 지출 많고 건강 목표도 설정된 상태)에서 성능/충돌 확인	-	전체 시나리오 통과, 성능 저하(캘린더 스와이프 렉 재발 등) 없음
38	최종 안정화: Day 36~37에서 발견된 문제 수정, STATUS.md 를 베타 배포 가능 상태로 최종 갱신, 커밋	STATUS.md	베타 착수 가능 상태 확정

7. 리스크 / 열린 질문

- 배달앱(요기요/배민) 공식 API: 개인 개발자 개방 여부 미확인 — Day 8에서 조사 후 딥링크 대안으로 스코프를 낮출 가능성 높음.
- 금융 API(토스 등): 마이데이터 사업자 등록 없이는 공식 연동 불가 가능성 높음 — be rich sir는 애초에 “수동 입력 + 분석”으로 스코프를 낮춤(Phase 3 설계에 반영됨).
- Cloudflare Workers AI 무료 티어 일일 한도: 정확한 실측치 미확인. be full/healthy/fun sir의 AI 추천 톨이 추가되면 호출량이 늘어나므로 Phase 5(Day 34)에서 실제 사용량 기준으로 재점검 필요.
- Workers AI 모델의 한국어 tool-calling 안정성: 현재 모델([mistral-small-3.1-24b-instruct](#))은 검증됐지만, Cloudflare가 모델을 폐지/교체할 수 있음(Gemini에서 실제로 겪은 일) — 모델 상수([WORKERS_AI_MODEL](#))가 죽으면 가장 먼저 확인할 지점.
- 1인 개발 기준이므로 Day 38까지는 “매일 온전히 이 작업만 했을 때” 가정. 실제 캘린더 일수는 더 길어질 수 있음(실제로 Phase 0는 예상보다 훨씬 오래 걸렸고 계획에 없던 Phase 0.5까지 발생함) — 순서를 지키는 것이 날짜를 지키는 것보다 중요하다. Phase당 코드 검사/실기기 테스트/안정화 3일이 추가되면서 전체 일수가 늘었지만,

이 3일은 건너뛰지 않는다 — 회귀를 놓치면 다음 Phase가 이전 Phase 위에 쌓이는 구조상 문제가 누적된다.

- ~~Claude 비전 지원 모델 확인, ShareExtension URL scheme 핸드오프 불확실성~~ — 둘 다 해소됨 (Workers AI 비전 지원 확인, App Group 파일 큐 방식으로 확정).